Introducción a la Dinámica Molecular

https://gemini.google.com/app/d0040f7d29692dcf

User prompt: quiero iniciar en modelara computacionalmente la dinamica molecular como le hago

Response: Para iniciar en la **dinámica molecular (DM)**, el camino principal consiste en comprender la física teórica de base (Mecánica Clásica aplicada a moléculas) y utilizar herramientas de software estándar en la industria e investigación.

1. Bases Teóricas Fundamentales

- **El Campo de Fuerza (Force Field):** Las moléculas no se modelan resolviendo la ecuación de Schrödinger a cada instante (sería demasiado pesado computacionalmente), sino mediante un **potencial clásico**.
  - **Términos enlazados:** Estiramiento de enlaces, ángulos de enlace y diedros (torsión).
  - **Términos no enlazados:** Interacciones electrostáticas (Ley de Coulomb) y van der Waals (Potencial de Lennard-Jones).
- **Integración Numérica:** Algoritmos como *Verlet* o *Leapfrog* para resolver $F = m \cdot a$ ($m\frac{d^2r}{dt^2} = -\nabla V(r)$) en pasos de tiempo extremada pequeños (del orden de **femtosegundos**, 10^{-15} s).
- **Ensembles Estadísticos:**
  - **NVE (Microcanónico):** Número de partículas, Volumen y Energía constantes.
  - **NVT (Canónico):** Control de temperatura mediante termostatos (ej. Nosé-Hoover, Berendsen).
  - **NPT (Isotérmico-Isobárico):** Control de presión y temperatura con barostatos (ej. Parrinello-Rahman).

2. Software Estándar para Iniciar

Software	Enfoque Principal	Lenguaje / Interfaz
GROMACS	Proteínas, membranas, biomoléculas. Extremadamente rápido en CPU/GPU.	C/C++, ejecutable por línea de comandos.
LAMMPS	Ciencia de materiales, polímeros, sistemas mecano-cuánticos reactivos.	C++, scripts personalizados.
OpenMM	Toolkit flexible de alto rendimiento optimizado para GPUs.	Python API / C++.
NAMD	Sistemas biológicos a gran escala.	Scripts Tcl / GUI VMD.

3. Flujo de Trabajo Típico (Workflow)

1. **Construcción del Sistema:** Obtener coordenadas iniciales (archivos `.pdb` o `.cif`).
2. **Solvatación e Ionización:** Agregar moléculas de agua (ej. TIP3P) y neutralizar la carga con iones (Na^+, Cl^-).
3. **Minimización de Energía:** Eliminar choques estéricos o traslapes atómicos.
4. **Equilibrado (NVT y NPT):** Estabilizar la temperatura y la densidad del sistema gradualmente.
5. **Producción (MD Run):** Simulación principal donde se extraen las trayectorias.
6. **Análisis:** Cálculo de RMSD (desviación cuadrática media), RMSF, radios de giro, enlaces de hidrógeno, o matrices de covarianza.

4. Stack Práctico Recomendado con Python

Para prototipar y aprender la física sin lidiar primero con compilaciones complejas, **OpenMM** combinado con **MDTraj** o **MDAnalysis** es la ruta más accesible:

```
# Ejemplo conceptual con OpenMM en Python
from openmm.app import *
from openmm import *
from openmm.unit import *

# 1. Cargar estructura y campo de fuerza
pdb = PDBFile('input.pdb')
forcefield = ForceField('amber14-all.xml', 'amber14/tip3pfb.xml')
```

```
# 2. Crear sistema
system = forcefield.createSystem(pdb.topology, nonbondedMethod=PME,
                                 nonbondedCutoff=1*nanometer, constraints=HBonds)

# 3. Configurar integrador y simulación
integrator = LangevinMiddleIntegrator(300*kelvin, 1/picosecond, 0.002*picoseconds)
simulation = Simulation(pdb.topology, system, integrator)
simulation.context.setPositions(pdb.positions)

# 4. Minimizar e integrar
simulation.minimizeEnergy()
simulation.step(10000) # 20 ps de simulación
```

User prompt: ¿Cómo instalo GROMACS con soporte CUDA en Ubuntu y ejecuto una simulación de prueba?

Response: callout-content-0

1 **Instalar dependencias del sistema**

Prerequisito

Abre una terminal y ejecuta las librerías necesarias para la compilación:

```
sudo apt update
sudo apt install -y build-essential cmake libfftw3-dev libxml2-dev python3-dev
```

Verifica que el compilador de C/C++ y CMake estén listos ejecutando `cmake --version`.

2 **Descargar y desempaquetar GROMACS**

5 min

Obtén el código fuente oficial de la última versión estable desde el repositorio:

```
wget https://ftp.gromacs.org/gromacs/gromacs-2024.2.tar.gz
tar xzf gromacs-2024.2.tar.gz
cd gromacs-2024.2
```

Confirma la descompresión revisando que exista el directorio `src/` dentro de la carpeta.

3 **Configurar y compilar con soporte CUDA**

10 - 20 min

Crea el directorio de compilación y ejecuta CMake activando el acelerador GPU:

```
mkdir build && cd build
cmake .. -DGMX_BUILD_OWN_FFTW=ON -DREGRESSIONTEST_DOWNLOAD=ON -DGMX_GPU=CUDA
make -j$(nproc)
sudo make install
source /usr/local/gromacs/bin/GMXRC
```

Para verificar que se compiló correctamente con soporte CUDA, ejecuta `gmx --version` y confirma que reconozca la GPU en la sección "Hardware detected".

4 **Ejecutar simulación de prueba (Lisozima en Agua)**

5 min

Descarga una proteína de prueba y prepara los archivos con el campo de fuerza AMBER99SB:

```
mkdir ~/prueba_gromacs && cd ~/prueba_gromacs
wget https://files.rcsb.org/download/1AKI.pdb
grep -v HOH 1AKI.pdb > 1AKI_clean.pdb

# Generar topología, caja y solvatación
gmx pdb2gmx -f 1AKI_clean.pdb -o 1AKI_processed.gro -water spce -ff amber99sb
gmx editconf -f 1AKI_processed.gro -o 1AKI_box.gro -c -d 1.0 -bt cubic
gmx solvate -cp 1AKI_box.gro -cs spc216.gro -o 1AKI_solv.gro -p topol.top
```

Verifica el archivo `1AKI_solv.gro` usando VMD, PyMOL o revisando las últimas líneas para confirmar que las moléculas de agua fueron agregadas.

5 **Minimización de energía en la GPU**

1 min

Crea un archivo de configuración mínimo (`minim.mdp`):

```
cat << 'EOF' > minim.mdp
integrator  = steep
emtol       = 1000.0
emstep      = 0.01
nsteps      = 50000
nstlist     = 1
cutoff-scheme = Verlet
ns_type     = grid
coulombtype = PME
rcoulomb    = 1.0
rvdw        = 1.0
pbc         = xyz
EOF

# Procesar y ejecutar en GPU
gmx grompp -f minim.mdp -c 1AKI_solv.gro -p topol.top -o em.tpr
gmx mdrun -v -deffnm em -nb gpu
```

Para confirmar el éxito de la simulación, verifica que el archivo `em.gro` se haya generado y que el valor de *Potential Energy* impreso al final sea negativo y convergente.

User prompt: ¿Cómo cargo el archivo de trayectoria de GROMACS (.xtc) en Python usando MDAnalysis para calcular el RMSD?

Response: Para analizar una trayectoria de GROMACS en Python, necesitas dos archivos principales:

1. **Archivo de topología/estructura:** `.gro` o `.tpr` (contiene los nombres de residuos, átomos y conectividad).
2. **Archivo de trayectoria:** `.xtc` o `.trr` (contiene las coordenadas atómicas a lo largo del tiempo).

A continuación se muestra el procedimiento paso a paso utilizando la librería **MDAnalysis**.

1. Instalación de dependencias

Si aún no tienes **MDAnalysis** y **Matplotlib** instalados en tu entorno de Python, puedes instalarlos mediante `pip` o `conda`:

```
pip install mdanalysis matplotlib
```

2. Script en Python para cargar la trayectoria y calcular el RMSD

Este script carga el sistema (*Universe*), selecciona los átomos de la proteína (C_α), alinea las estructuras de cada fotograma (*frame*) contra la referencia inicial para eliminar traslaciones/rotaciones de cuerpo rígido y calcula la Desviación Cuadrática Media (RMSD).

```
import MDAnalysis as mda
from MDAnalysis.analysis import rms
import matplotlib.pyplot as plt
import numpy as np

# 1. Cargar la topología y la trayectoria
# Usa el archivo .gro (o .tpr) de la estructura de referencia y el .xtc de la trayectoria
topology_file = "1AKI_solv.gro"   # O tu archivo de referencia/equilibrado
trajectory_file = "em.xtc"        # O la trayectoria de producción (.xtc)

u = mda.Universe(topology_file, trajectory_file)

print(f"Número de fotogramas (frames): {len(u.trajectory)}")
print(f"Número total de átomos: {len(u.atoms)}")

# 2. Configurar la herramienta de análisis de RMSD
# Seleccionamos 'backbone' o 'name CA' para evaluar el plegamiento estructural
rmsd_analysis = rms.RMSD(
    u,                    # El universo con la trayectoria
    u,                    # El mismo universo como referencia (frame 0 por defecto)
    select="name CA",     # Selección para alineación y cálculo (átomos C-alpha)
```

```
    groupselections=["protein"], # Selección adicional (ej. todos los átomos de la proteína)
    ref_frame=0             # Fotograma de referencia
)

# 3. Ejecutar el cálculo a lo largo de toda la trayectoria
rmsd_analysis.run()

# 4. Extraer los resultados
# rmsd_analysis.results.rmsd contiene una matriz con las columnas:
# [Frame, Tiempo (ps), RMSD_CA (Å), RMSD_protein (Å)]
rmsd_data = rmsd_analysis.results.rmsd

frames = rmsd_data[:, 0]
time_ps = rmsd_data[:, 1]
rmsd_ca = rmsd_data[:, 2]      # RMSD alineado solo con C-alpha
rmsd_protein = rmsd_data[:, 3] # RMSD del grupo completo de la proteína

# 5. Graficar los resultados
plt.figure(figsize=(8, 5))
plt.plot(time_ps, rmsd_ca, label=r"C-$\alpha$", color="navy", linewidth=1.5)
plt.plot(time_ps, rmsd_protein, label="Proteína Completa", color="crimson", alpha=0.7, linestyle="--")

plt.xlabel("Tiempo (ps)", fontsize=12)
plt.ylabel(r"RMSD ($\AA$)", fontsize=12)
plt.title("Evolución del RMSD a lo largo de la Trayectoria", fontsize=14)
plt.grid(True, linestyle=":", alpha=0.6)
plt.legend(loc="best")
plt.tight_layout()

# Guardar la gráfica
plt.savefig("rmsd_plot.png", dpi=300)
plt.show()
```

Puntos clave a considerar:

- **Manejo de Condiciones Periódicas de Frontera (PBC):** Si la proteína "rompe" la caja o se salta de un extremo a otro durante la simulación de producción, procesa primero la trayectoria en GROMACS antes de cargarla en Python:

  ```
  gmx trjconv -s prod.tpr -f prod.xtc -o prod_noPBC.xtc -pbc mol -center
  ```

- **Selección de Átomos:** Puedes cambiar `select="name CA"` por `select="backbone"` (esqueleto peptídico) o `select="protein"` según el detalle geométrico que requieras evaluar.
